

Hybrid RAG Pipeline

Dense + Sparse Retrieval with RRF Fusion

RAG-Based Energy Forecasting

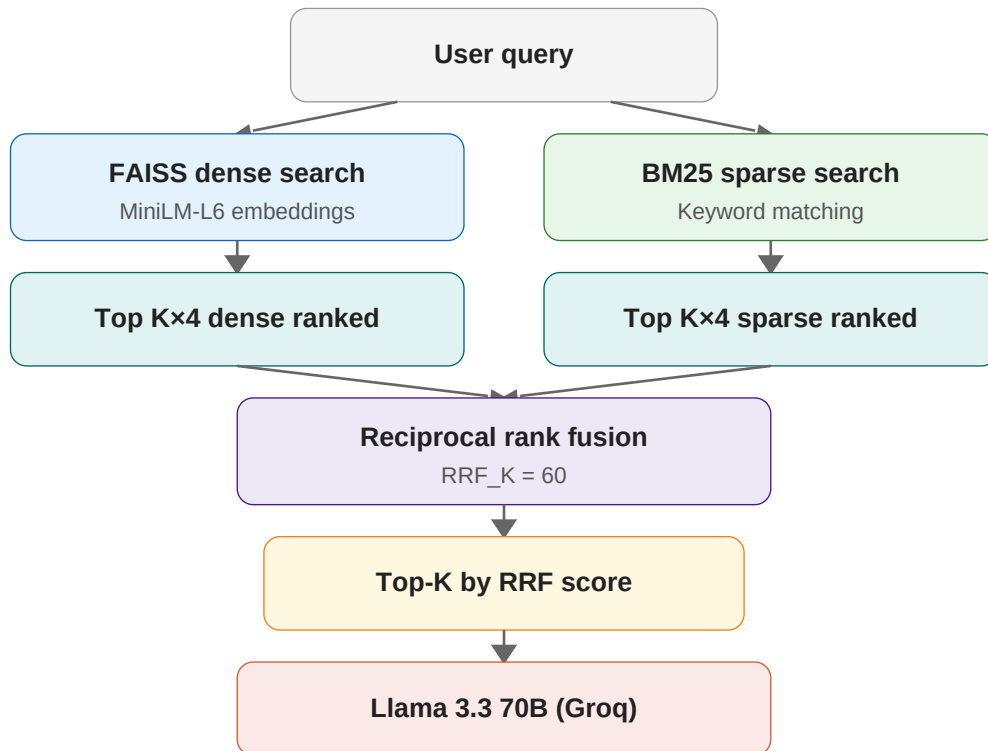
MSc Thesis — LJMU

Zoheb Anwar Hussain

1. Overview

The Hybrid RAG pipeline (EXP_03) combines FAISS dense retrieval with BM25 sparse retrieval, fusing results via Reciprocal Rank Fusion (RRF). This addresses a key limitation of dense-only retrieval: the lexical mismatch problem where conceptual queries (e.g., 'peak demand season') fail to match numeric KB chunks (e.g., 'Zone 18 saw 241734.8 MW').

2. Architecture Diagram



3. Architecture Components

3.1 Dense Retrieval (FAISS)

Identical to EXP_02. Query is embedded via all-MiniLM-L6-v2 and searched against the FAISS index. Returns top-K*4 candidates (pool size) for RRF fusion.

3.2 Sparse Retrieval (BM25)

BM25Okapi index built from tokenised chunk texts at initialisation. Query is tokenised and scored against all chunks. Returns top-K*4 candidates ranked by BM25 score. Captures exact keyword matches that dense retrieval misses.

3.3 Reciprocal Rank Fusion (RRF)

RRF combines the two ranked lists using the formula: $RRF_score(d) = 1/(K_rrf + rank_dense(d)) + 1/(K_rrf + rank_sparse(d))$, where $K_rrf = 60$ (standard constant). Documents appearing in both lists get boosted scores. Final top-K documents selected by descending RRF score.

4. Why Hybrid Retrieval?

The energy forecasting KB has a fundamental lexical mismatch: user queries use conceptual language while KB chunks contain numeric statistical data. Dense retrieval captures semantic similarity but misses exact zone/date matches. BM25 captures keyword overlap but misses semantic meaning. RRF fusion leverages both signals, improving ranking quality (MRR) without changing recall significantly.

5. Key Configuration

Parameter	Value	Purpose
RRF_K	60	Standard RRF constant
Pool size	$K \times 4$	Candidate pool for each retriever
Tokenisation	<code>lower().split()</code>	BM25 word tokenisation
K values	3, 5, 10	Retrieval depth tested

6. Key Files

File	Purpose
<code>experiments/exp_03_hybrid_rag.py</code>	HybridRetriever class + experiment
<code>rank_bm25</code> (library)	BM25Okapi sparse scoring
<code>src/embedding/faiss_store.py</code>	FAISS dense index